

Manual Técnico del Frontend

Sistema de Gestión de Historias Clínicas

Tabla de contenidos

1. Introducción	3
1.1. Propósito	3
2. Arquitectura del frontend	4
2.1. Visión general	4
2.2. Estructura lógica esperada	4
2.3. Principios de diseño	5
3. Tecnologías	6
3.1. Stack base	6
3.2. Estado actual del módulo analizado	6
4. Integración con API REST	7
4.1. Convención de consumo	7
4.2. Reglas de integración	7
4.3. Contrato de respuesta	7
5. Seguridad JWT	9
5.1. RNF-01	9
5.2. Flujo mínimo	9
5.3. Regla Zero Trust	9
6. Flujo de datos entre React y Django	10
6.1. Secuencia funcional	10
6.2. Responsabilidades por capa	10
7. Mapa de navegación	11
7.1. Rutas públicas	11
7.2. Rutas protegidas	11
7.3. Navegación global	11
8. Estándares de UI	12
8.1. Principios	12
8.2. Estados visuales obligatorios	12
9. Testing y calidad	13
9.1. Estrategia	13
9.2. Criterios de calidad	13

10.Convenciones de citación técnica	14
10.1. Reglas documentales	14
10.2. Buenas prácticas	14
11.Cierre	15

Capítulo 1

Introducción

Este manual describe la arquitectura técnica del frontend del sistema de gestión de historias clínicas desarrollado en React.

La documentación se basa en dos referencias principales:

- El análisis del módulo **MediCampus-FrontEnd**, que muestra la base real del frontend.

1.1. Propósito

- Explicar la estructura técnica de la SPA.
- Documentar la integración con la API REST.
- Describir la autenticación JWT requerida por RNF-01.
- Establecer criterios de organización por componentes.

Capítulo 2

Arquitectura del frontend

2.1. Visión general

El frontend se concibe como una **SPA (Single Page Application)** construida con React, Vite y TypeScript.

La arquitectura debe seguir el principio de **Packaging by Component**, separando responsabilidades en:

- **views/**: composición de pantallas.
- **services/**: consumo de API y lógica de acceso a datos.
- **hooks/**: orquestación de estado y reglas de interacción.
- **components/**: componentes reutilizables y componentes específicos del módulo.

2.2. Estructura lógica esperada

```
src/  
  app/  
    router/  
    providers/  
  modules/  
    dashboard/  
    agendas/  
    seguridad/  
  shared/  
    api/  
    types/  
    utils/  
  ui/
```

```
components/  
theme/  
globals.css
```

2.3. Principios de diseño

- La lógica de negocio no debe residir en las vistas.
- Los componentes visuales deben ser “dumb components”.
- Los servicios deben ser reutilizables y desacoplados del render.
- El frontend solo consume y presenta estados; el backend concentra el negocio.

Capítulo 3

Tecnologías

3.1. Stack base

- **React**: interfaz de usuario mediante componentes funcionales y hooks.
- **Vite**: bundler y entorno de desarrollo.
- **TypeScript**: tipado estático.
- **Vitest**: pruebas unitarias y de comportamiento.
- **React Testing Library**: validación de interacción y comportamiento del usuario.

3.2. Estado actual del módulo analizado

El análisis del módulo `MediCampus-FrontEnd` muestra un scaffold inicial con:

- `src/main.tsx` como punto de arranque.
- `src/App.tsx` concentrando estado local demo.
- `src/api.ts` centralizando el consumo HTTP inicial.
- `src/api.test.ts` como base de pruebas.

Capítulo 4

Integración con API REST

4.1. Convención de consumo

Todos los servicios del frontend deben consumir endpoints con prefijo:

```
/api/v1/
```

4.2. Reglas de integración

- No establecer dominios en componentes.
- Centralizar la URL base en una capa común.
- Encapsular el acceso HTTP en servicios.
- Mantener consistencia en el contrato de respuestas.

4.3. Contrato de respuesta

El frontend debe esperar respuestas tipo:

```
{  
  "success": true,  
  "message": "...",  
  "data": {}  
}
```

Y para errores:

```
{  
  "success": false,  
  "message": "...",  
  "errors": {
```

```
    "detail": "..."  
  }  
}
```

Capítulo 5

Seguridad JWT

5.1. RNF-01

La aplicación debe validar la existencia y vigencia del token JWT antes de:

- acceder a rutas protegidas,
- ejecutar peticiones privadas,
- mostrar información sensible.

5.2. Flujo mínimo

1. Inicio de sesión.
2. Recepción de `access` y `refresh`.
3. Almacenamiento local del token.
4. Envío de `Authorization: Bearer <token>`.
5. Renovación o cierre de sesión cuando corresponda.

5.3. Regla Zero Trust

Si el token no existe o no es válido:

- limpiar la sesión,
- redirigir al login,
- impedir el acceso a la navegación protegida.

Capítulo 6

Flujo de datos entre React y Django

6.1. Secuencia funcional

1. El usuario interactúa con una vista.
2. La vista delega la operación a un hook.
3. El hook llama a un servicio.
4. El servicio consume la API Django.
5. Django responde en JSON.
6. El hook normaliza el resultado.
7. La vista renderiza estados de carga, vacío, error o éxito.

6.2. Responsabilidades por capa

- **Vista:** composición y presentación.
- **Hook:** estado e acción.
- **Servicio:** HTTP y transformación de datos.
- **Backend:** lógica de negocio y persistencia.

Capítulo 7

Mapa de navegación

7.1. Rutas públicas

- /login

7.2. Rutas protegidas

- /dashboard
 - resumen institucional,
 - indicadores principales.
- /agendas
 - gestión de citas,
 - seguimiento de atenciones.
- /seguridad
 - acceso,
 - perfil,
 - control de sesión.

7.3. Navegación global

- Header con contexto de sesión.
- Sidebar o menú lateral por módulo.
- Redirección por defecto a /dashboard después de autenticación.

Capítulo 8

Estándares de UI

8.1. Principios

- Interfaz responsive para móvil, tablet y escritorio.
- Uso de tokens y utilidades del sistema visual.
- Reutilización de componentes base desde `src/ui/components`.
- Creación de componentes específicos solo cuando el módulo lo requiera.

8.2. Estados visuales obligatorios

- `loading`
- `empty`
- `error`
- `success`

Capítulo 9

Testing y calidad

9.1. Estrategia

La documentación técnica debe reflejar que el frontend se valida con:

- Vitest,
- React Testing Library,
- pruebas orientadas al comportamiento del usuario.

9.2. Criterios de calidad

- TDD como práctica obligatoria.
- Sin lógica de negocio en vistas.
- Servicios desacoplados y reutilizables.
- Tipado estricto de DTOs y respuestas.

Capítulo 10

Convenciones de citación técnica

10.1. Reglas documentales

La documentación debe citar rutas y símbolos técnicos cuando describa decisiones de arquitectura.

Ejemplos:

- `src/main.tsx`
- `src/App.tsx`
- `src/api.ts`
- `spec_kit/constitution.md`

10.2. Buenas prácticas

- Mantener trazabilidad entre la especificación y el manual técnico.
- Usar términos consistentes a lo largo del documento.
- Diferenciar claramente entre estado actual y arquitectura objetivo.

Capítulo 11

Cierre

Este archivo sirve como plantilla inicial del manual técnico del frontend y puede ampliarse conforme avance la implementación real.